# Homework 2 Simulation

Calvin Chang **all code was written in python*

## Problem 1

**a)** Given that $X \sim Bin(n, p)$ we know that:

$$\begin{aligned} p(k) &= \binom{n}{k} p^k (1-p)^{n-k} \\ p(k+1) &= \binom{n}{k+1} p^{k+1} (1-p)^{n-k-1} \end{aligned} \tag{1}$$

We can simplify the term:

$$\binom{n}{k+1} = \frac{n-k}{k+1} \binom{n}{k} \tag{2}$$

Which we then bring back to find and prove that:

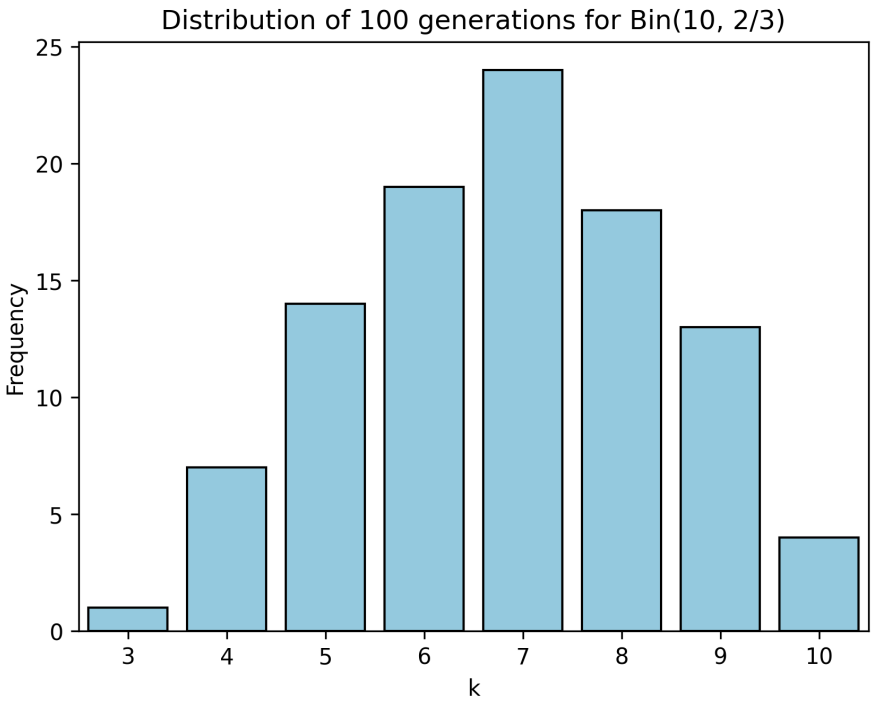
$$\begin{aligned} p(k+1) &= \binom{n}{k+1} p^{k+1} (1-p)^{n-k-1} \\ &= \frac{n-k}{k+1} \binom{n}{k} p^{k+1} (1-p)^{n-k-1} \\ &= \frac{n-k}{k+1} \frac{p}{1-p} \binom{n}{k} p^{k} (1-p)^{n-k} \\ p(k+1) &= \frac{n-k}{k+1} \frac{p}{1-p} p(k), k = 1, 2, 3, \dots n-1 \end{aligned} \tag{3}$$

**b)** To use the inverse transform method to generate $X$, we need to use **a)** so that we dont have to calculate the $\binom{n}{k}$ for every iteration. Based on a), we can use the previous iteration value to generate the next value, then iterate until we have X. Therefore there is pseudo code for the given values `n` , `p`

```
U = rand(0,1)           # generate uniform distribution 0~1
k = 0                   # initialize
pmf = (1-p)*n           # p(0)
cdf = pmf               # F(0)

while U > cdf:
    k += 1
    pmf = pmf * ((n-k-1)/k) * (p/(1-p))
    cdf += pmf
return k
```

**c)**



## Problem 2

**a)** We are given:

$$
\begin{aligned}
p(k) &= \binom{k-1}{r-1} p^r (1-p)^{\,k-r}, \quad k = r, r+1, \dots \\
p(k+1) &= \binom{k}{r-1} p^r (1-p)^{\,k-r+1}
\end{aligned} \tag{4}
$$

To find the result, we then calculate:

$$
\begin{aligned}
\frac{p(k+1)}{p(k)} &= \frac{\binom{k}{r-1} p^r (1-p)^{\,k-r+1}}{\binom{k-1}{r-1} p^r (1-p)^{\,k-r}} \\
&= \frac{\binom{k}{r-1}}{\binom{k-1}{r-1}} \cdot (1-p)
\end{aligned} \tag{5}
$$

Now we need to simplify:

$$
\begin{aligned}
\frac{\binom{k}{r-1}}{\binom{k-1}{r-1}} &= \frac{k!}{(r-1)!(k-r+1)!} \cdot \frac{(r-1)!(k-r)!}{(k-1)!} \\
&= \frac{k}{k-r+1}
\end{aligned} \tag{6}
$$

Now to bring back into the original function:

$$
\begin{aligned}
\frac{p(k+1)}{p(k)} = \frac{\binom{k}{r-1}}{\binom{k-1}{r-1}} \cdot (1-p) &= \frac{k}{k-r+1} \cdot (1-p) \\
\frac{p(k+1)}{p(k)} &= \frac{k(1-p)}{k-r+1} \\
p(k+1) &= \frac{k(1-p)}{k-r+1} p(k)
\end{aligned} \tag{7}
$$

**b)** Based on part **a)**, the pseudo code for $NB(r,p)$ for given values `n` and `p` is:

```
function NB(r,p):
    U = rand(0,1)           # generate uniform distribution 0~1
    k = r                   # initialize
    pmf = p**r              # p(0)
    cdf = pmf               # F(0)

    while U > cdf:
        pmf = pmf * (k*(1-p)/(k-r+1))
        k += 1
        cdf += pmf
    return k
```

**c)** Since $NB(r,p)$ represents the number of Bernoulli trials to get $r$ successes, and $Geom(p)$ represents the number of Bernoulli trials to get 1 sucess, then the relationship between $NB(r,p)$ and $Geom(p)$ can be defined as:

$$NB(r,p) = \sum_{i=0}^{r} G_i \tag{8}$$

For Geometric distributions, we also know:

$$G = 1 + \left\lfloor \frac{\ln(1-U)}{\ln(1-p)} \right\rfloor. \tag{9}$$

Therefore, the pseudo code for $Geom(p)$ can be written for value `p` as:

```
function geom(p):
    U = rand(0,1)                                   # Uniform(0,1)
    result = 1 + floor(log(1 - U) / log(1 - p))     # Function for G
    return result
```

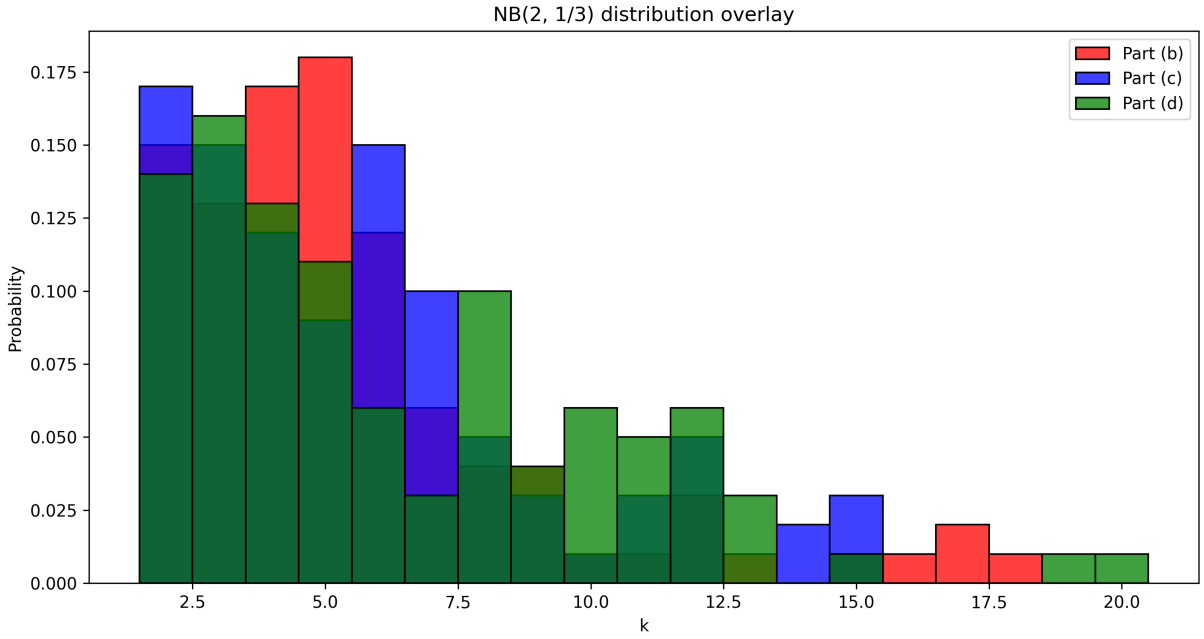
This can then be incorporated into a new $NB(r, p)$ function that is:

```
function NB(r, p):
    k = 0
    for i in range(r):
        G = geom(p)
        k += G
    return k
```

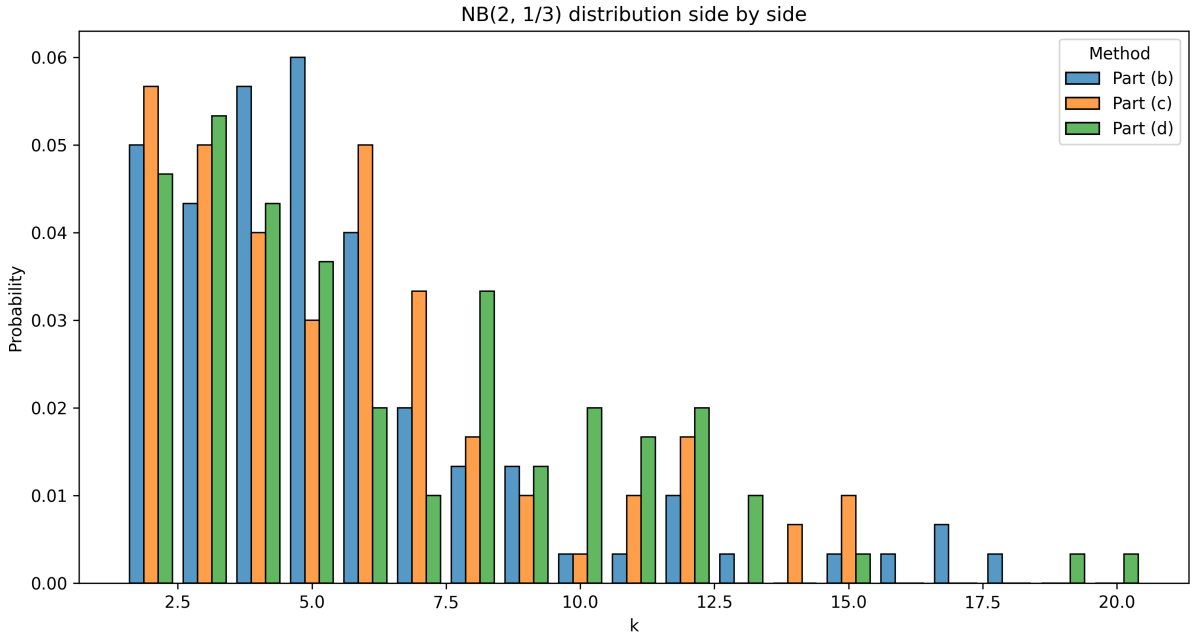
**d)** Based on the definition of the Bernoulli trials, we can create a new function $NB(r, p)$ that has pseudocode:

```
function NB(r, p):
    success = 0
    trials = 0
    while success < r:
        U = rand(0,1)
        if U < p:
            success += 1
        trials += 1
    return trials
```

**e)** Distribution graph with each bar overlayed:




Distribution graph with each bar side by side:



## Problem 3

From the given function:

$$\Pr(X = j) \;=\; \left(\tfrac{1}{2}\right)^{j+1} \;+\; \left(\tfrac{1}{2}\right)\frac{2^{\,j-1}}{3^{\,j}}, \qquad j = 1, 2, \ldots \tag{10}$$

Its clear that $(\frac{1}{2})^{j+1}$ is the geometric distribution for $Geometric(1/2)$ multiplied by $\frac{1}{2}$, represent that as $Y \sim Geom(1/2)$ Then observing the second part of $Pr(X = j)$, we can simplify for:

$$(\frac{1}{2})\frac{2^{j-1}}{3^j} = \frac{1}{2} \cdot \frac{1}{3} \cdot \frac{2^{j-1}}{3^{j-1}} \tag{11}$$

Which we can observe is also just the geometric distribution for $Geometric(1/3)$ multiplied by $\frac{1}{2}$. And we can again represent that as $Z \sim Geom(1/3)$ So the entire function can be simplified to:

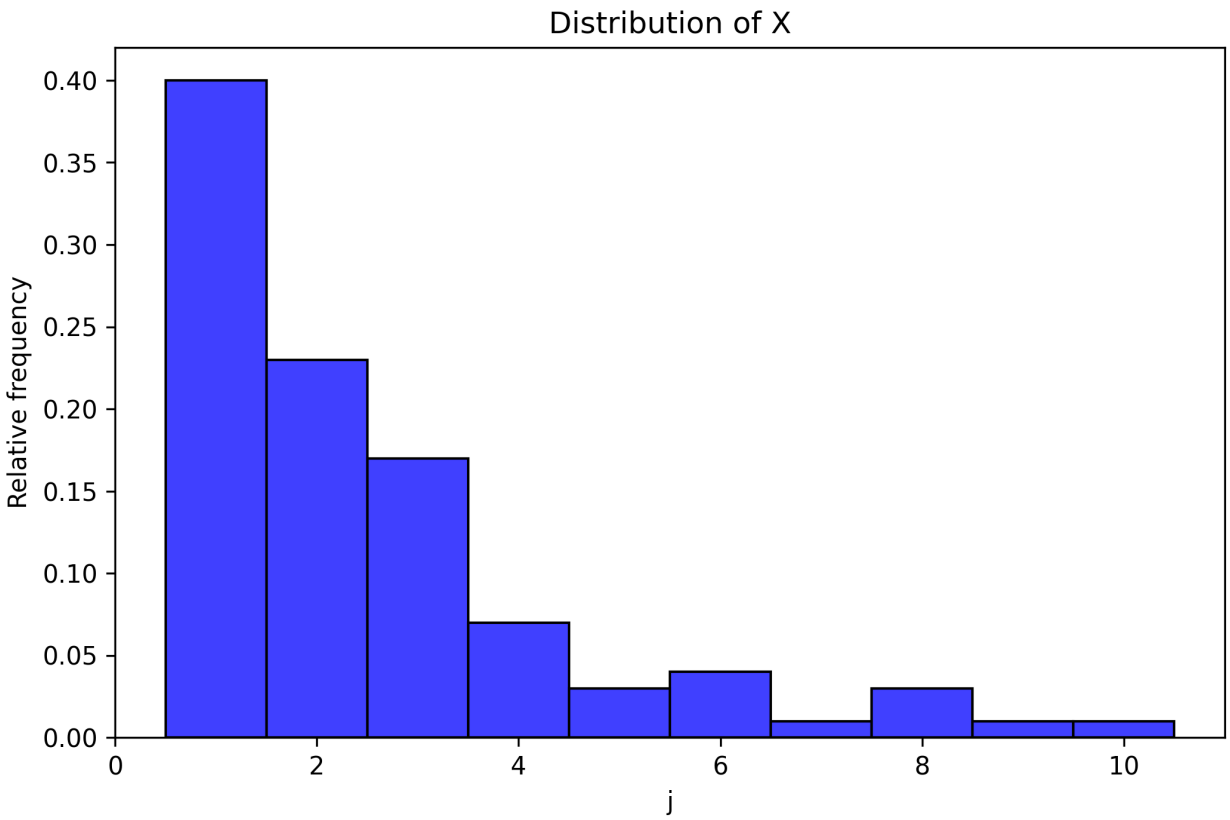
$$Pr(X = j) = \frac{1}{2}P(Y = j) + \frac{1}{2}P(Z = j) \tag{12}$$

This represents a $50\%$ chance for $P(Y = j)$ and a $50\%$ chance for $P(Z = j)$. Therefore we can write pseudo code to represent this as:

```
function geom(p):
    U = random.uniform(0,1)
    return 1 + math.floor(math.log(1 - U) / math.log(1 - p))

function X():
    # represent the 50% chance for either P()
    flip = random.uniform(0,1)
    if flip < 0.5:
        return geom(1/2)
    else:
        return geom(1/3)
```

Based on the pseudo code, a python code was created and after 100 runs this was the distribution:

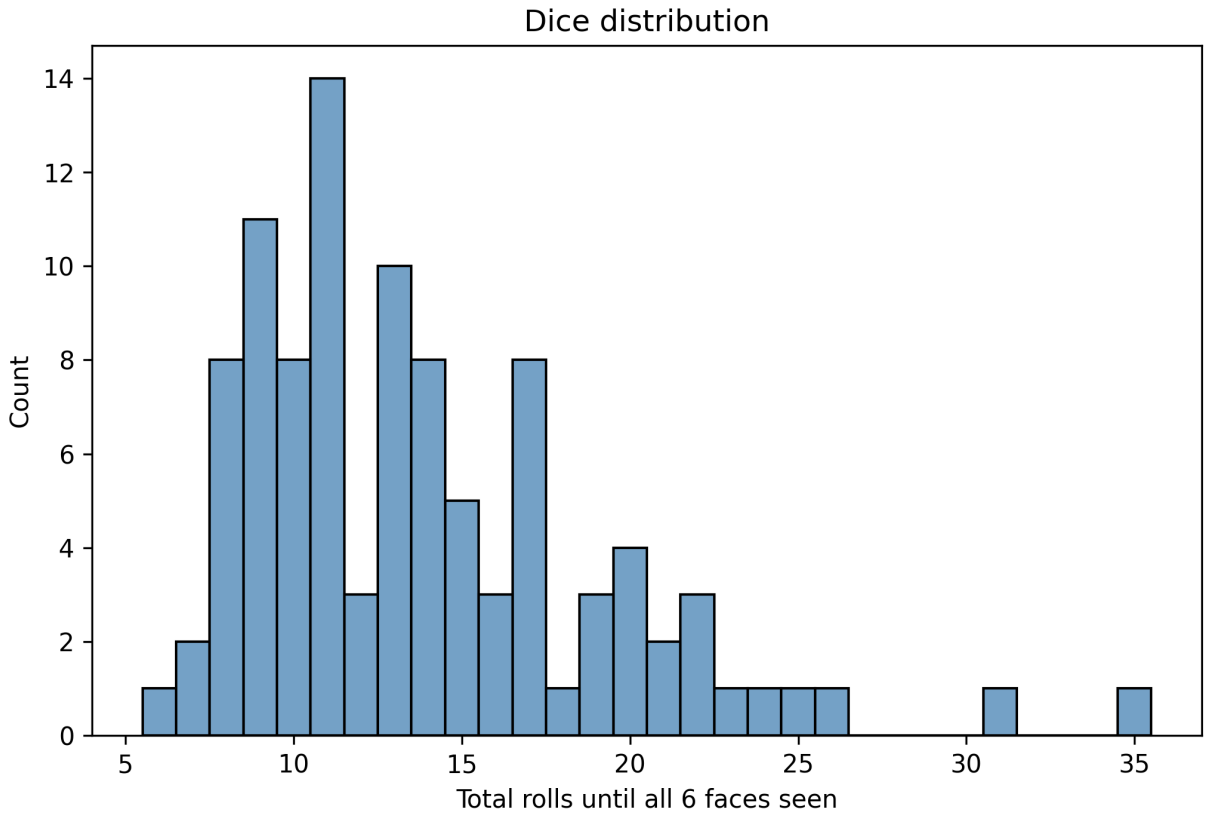


## Problem 4

Based on the problem, there is pseudocode:

```
function roll_dice():
    seen = set                              # save all seen values
    rolls = 0                               # number of rolls
    while size(seen) < 6:
        dice_roll = random.random(1,6)      # a random dice roll
        seen.add(dice_roll)
        rolls += 1
    return rolls
```

Now given this function, we can implement it within python and run it 100 times and plot the distribution to get the graph for 100 rolls:



And also 1000 rolls: